

# AI Homework 3

112550184 陳璽安

## 1. CNN

### Work Flow:

- 1) used load\_train\_dataset to load training dataset(images and true labels) and used load\_test\_dataset to load testing dataset(only images).
- 2) Create TrainDataset and TestDataset and package them by DataLoader.
- 3) For every epoch we load TrainDataset into train() function to train the model and call validate() function to evaluate the model accuracy.
- 4) Test the model by load TrainDataset into test() function, and store the predicted results in CNN.csv.

### Code Explanation

#### util.py

1. load\_train\_dataset(...) → load train dataset ( image paths and labels)  
Specified the labels

```
label = {  
    'elephant':0,  
    'jaguar':1,  
    'lion':2,  
    'parrot':3,  
    'penguin':4  
}
```

For each folder(animal) in train folder, store image path and image label in images and labels and return.

```
images = []  
labels = []  
  
for animal in os.listdir(path):  
    p = os.path.join(path, animal)  
    if not os.path.isdir(p):  
        continue  
    l=label.get(animal)  
  
    for image in os.listdir(p):  
        image_path = os.path.join(p,image)  
        if os.path.isfile(image_path):  
            images.append(image_path)  
            labels.append(l)  
  
return images, labels
```

2. load\_test\_dataset(...) → load test dataset ( only image path)

For each image in test folder, store the image path in images and return.

```
images = []
for image in os.listdir(path):
    image_path = os.path.join(path, image)
    if os.path.isfile(image_path):
        images.append(image_path)
return images
```

## CNN.py

### class CNN(nn.Module)

1. **def \_\_init\_\_(...)** → Used three convolution block, each block consists of convolution layer + batch normalization + ReLU activation + max polling. After convolutions layers, a fully connected layer maps to 256 neurons, and a dropout layer to preventing overfitting. Finally, a fully connected layer output the classification score for 5 classes.

Initialize CNN with parent module(nn.Module) initialization.

```
super(CNN, self).__init__()
```

First convolution layer takes 3 input channels(RGB), and output 32 feature maps. And first batch normalization layer normalize the 32 feature maps.

```
self.conv1 = nn.Conv2d(3,32,3,1)
self.bn1 = nn.BatchNorm2d(32)#normalize data
```

Activation function and max polling layer(with 2\*2 kernel).

```
self.relu = nn.ReLU()
#kernel_size
self.pool = nn.MaxPool2d(2)
```

Second convolution layer maps from 32 channels to 64 channels. And second batch normalization layer normalize the 64 feature maps.

```
self.conv2 = nn.Conv2d(32,64,3,1)
self.bn2 = nn.BatchNorm2d(64)
```

Third convolution layer maps from 64 channels to 128 channels. And third batch normalization layer normalize the 128 feature maps.

```
self.conv3 = nn.Conv2d(64, 128,3,1)
self.bn3 = nn.BatchNorm2d(128)
```

First fully connected layer that takes the flattened output from the convolutional layers (128 channels of 26×26 feature maps) and maps it to 256 neurons, and dropout layer randomly deactivates a specified rate of neurons during training to prevent overfitting. Final fully connected layer that maps 256 features to the 5 output classes for classification.

```
self.fc1 = nn.Linear(128*26*26, 256)
self.dropout = nn.Dropout(0.3)
self.fc2 = nn.Linear(256, num_classes)
```

## 2. def forward(self, x)

Go through first, second and third convolution blocks.(convolution layer → batch normalization → ReLU activation function → max pooling )

```
#conv1,2,3
x = self.pool(self.relu(self.bn1(self.conv1(x))))
x = self.pool(self.relu(self.bn2(self.conv2(x))))
x = self.pool(self.relu(self.bn3(self.conv3(x))))
```

Flatten the output and go through the fully connected layer. Applied ReLU activation and dropout function with 30% dropout rate. Finally, the second fully connected layer produced the classification output.

```
x=x.view(x.size(0),-1)
x=self.fc1(x)
x=self.relu(x)
x=self.dropout(x)
x=self.fc2(x)
```

## Other functions

### 1. def train(...) → train the model with train dataset and return the average loss

Set the model into train mode, and initialize the sample number(n) and total loss to be zero.

```
model.train()

all_loss=0.0
n=0
```

Iterate each image path and label in train dataset. Load image and label to GPU(or CPU), and clear the existing gradient.

```
for images, labels in tqdm(train_loader, desc='training'):
    images = images.to(device)
    labels = labels.to(device)

    optimizer.zero_grad()#清空梯度
```

Predict the label of the image, and calculate the loss between predicted result and the real label.

```
outputs = model(images)
loss = criterion(outputs, labels)
```

Updates the model parameters based on the calculated gradients using the optimizer's update rule.

```
loss.backward()#計算 loss 的梯度
optimizer.step()#根據梯度更新模型參數
```

Track the total loss and the total number of samples.

```
num = images.size(0)
all_loss += loss.item()*num #.item return scalar
n+=num
```

Get average loss by dividing total loss to total sample number, and return average loss.

```
avg_loss = all_loss/n
return avg_loss
```

## 2. **def validate(...)** → validate the model and return the average loss and accuracy

Set the model to evaluation mode and initialize total loss, correct predict number, and all sample number to be zero.

```
model.eval()#evaluation mode

all_loss = 0.0
correct = 0#correct prediction
n=0#all sample num
```

Do not calculate gradient in the process, and iterate image path and label in train dataset.  
Load image and label to GPU(or CPU).

```
with torch.no_grad():
    for images, labels in tqdm(val_loader, desc='validate'):
        images = images.to(device)
        labels = labels.to(device)
```

Calculate loss and find the predicted result. Track the total sample size and total loss. And if the predict result is correct, increase the correct result number.

```
outputs = model(images)#為 batch_size*class num 的 tensor
loss = criterion(outputs, labels)

_, predicted = torch.max(outputs, 1)#找出每個 output 預測最大值的 label 的 index

num=images.size(0)#batch size
all_loss+=loss.item()*num
correct+=(predicted==labels).sum().item()
n+=num
```

Get average loss and accuracy(correct result number/total sample number)

```
avg_loss = all_loss/n
accuracy = correct/n
return avg_loss, accuracy
```

### 3. def test(...) → test the model with test dataset and write the result to 'CNN.csv'

Set the model to evaluation mode, and initialize predicted result and image IDs to be empty list.

```
model.eval()#evaluation mode
```

```
predict = []
```

```
imageID = []
```

Do not calculate gradient in the process. Iterate data in testdataset, and get the predicted result(label) of each image.

```
with torch.no_grad():  
    for images, ids in tqdm(test_loader, desc = 'test'):  
        images = images.to(device)  
        outputs = model(images)  
        _, predicted = torch.max(outputs,1)
```

Store the predicted result in predict list and record the image ID in list imageID.

```
predict.extend(predicted.cpu().numpy())
```

```
imageID.extend(ids)
```

Turn the result into dataframe and convert it into csv file.

```
results = {'id':imageID, 'prediction':predict}
```

```
df = pd.DataFrame(results)
```

```
df.to_csv('CNN.csv', index = False)
```

## 2. Decision Tree

### Work Flow:

- 1) used `load_train_dataset` to load training dataset(images and true labels) and used `load_test_dataset` to load testing dataset(only images).
- 2) Create `TrainDataset` and `TestDataset` and package them by `DataLoader`.
- 3) Load the pretrained `ConvNet` model for feature extraction.
- 4) Extract features from training images by using `get_features_and_labels` function.
- 5) Train the Decision Tree model with extracted features by calling `tree_model.fit()`, which builds the tree recursively using information gain as the splitting criterion.
- 6) Extract features from test dataset by using `get_features_and_paths` function, and make predictions on test features using `predict` function. Calculate accuracy of predicted result and print validation accuracy.
- 7) Save the predictions to `DecisionTree.csv` with image IDs

### Code Explanation:

#### Feature Extraction

1. `def get_features_and_labels(...)` → Use `ConvNet` to extract feature, and return the features and the labels.

Set the model into evaluation mode, and initialize the feature list and label list to be empty list.

```
model.eval()#evaluation mode
features = []
labels = []
```

Don't calculate gradient, and iterate each batch of data (images and labels) in `DataLoader`. Load image into model to get feature and store them in feature list. Store the label in the label list.

```
with torch.no_grad():#don't track gradient
    for images, label in tqdm(dataloader, desc = 'extracting features and labels'):
        images = images.to(device)
        feature = model(images)
        features.extend(feature.cpu().numpy())
        labels.extend(label.cpu().numpy())
```

Convert features into dataframe format and return features and labels.

```
features = pd.DataFrame(features)

return features, labels
```

## 2. `def get_features_and_paths(...)` → Extract feature by CovNet and return features and image paths.

Set the model into evaluation mode, and initialize features list and paths list into empty lists.

```
model.eval()#evaluation mode
features = []
paths = []
```

Iterate each batch of data in DataLoader, and load them into CovNet model to get features. Store the features and image paths in the features list and paths list.

```
with torch.no_grad():
    for images, path in tqdm(data_loader, desc = 'extracting features and paths'):
        images = images.to(device)
        feature = model(images)
        features.extend(feature.cpu().numpy())
        paths.extend(path)
```

Convert features into dataframe format and return features and paths.

```
features = pd.DataFrame(features)
return features, paths
```

## Model Architecture

### 1. `def _build_tree(...)` → Recursively build decision tree

Get total sample number and feature number. ( $X \text{ size} = \text{sample number} * \text{feature number}$ )

Get class number.

```
sample_num, feature_num = X.shape
class_num = len(np.unique(y))
```

Stopping condition: reach the max depth or here's only one class or the sample number is less than two. If we reach the stopping condition, return the leaf value as the class with highest occurrence rate and set the node type as leaf.

```
if(depth>=self.max_depth or class_num==1 or sample_num<2):
    leaf_val = np.argmax(np.bincount(y))#找到 y 中出現最多次的 class
    self.progress.update(1)#update 進度條
    return{'type': 'leaf', 'value':leaf_val}
```

If it's not stopping condition, get the best split feature index and split threshold by `_best_split` function.

```
idx, thres = self._best_split(X,y)
```

If there's no best split → it's a leaf node. Return the leaf value(class with highest occurrence) and set node type as leaf.

```

if idx is None:
    val = np.argmax(np.bincount(y))
    self.progress.update(1)
    return {'type': 'leaf', 'value': val}

```

If there exists best split, split the data with `_split_data` function.

```

xl, yl, xr, yr = self._split_data(X, y, idx, thres)

```

If one side of the data is null → it's leaf node. Return leaf value(class with highest occurrence) and set the node type as leaf.

```

if len(xr)==0 or len(xl)==0:
    val = np.argmax(np.bincount(y))
    self.progress.update(1)
    return {'type': 'leaf', 'value': val}

```

Build left subtree and right subtree by recursively calling `_build_tree` function with splitted data.

```

left_subtree = self._build_tree(xl, yl, depth+1)
right_subtree = self._build_tree(xr, yr, depth+1)

```

Return the best split feature index , split threshold, left subtree, right subtree and set the node type as 'node'.

```

return {
    'type': 'node',
    'feature_idx': idx,
    'threshold': thres,
    'left': left_subtree,
    'right': right_subtree
}

```

## 2. `def _split_data (...)` → Split the data by the given feature index and threshold

Create a mask to select all data that the feature value is less than threshold.

```

mask = X.iloc[:, feature_index] <= threshold

```

Split data into two side, the left side contain the data that has the feature value that is less than threshold.

```

left_X = X[mask]
left_y = y[mask_np]

```

The right side contain the data has feature value more than threshold.

```

right_X = X[~mask]
right_y = y[~mask_np]

```



3. **def \_best\_split(...)** → find the best split that will minimize the entropy for the data  
Initialize best threshold and feature index for split to be zero, and lset best information gain to be negative infinity. Compute entropy of current data, and get total sample number.

```
best_threshold=None
best_feature_index=None
best_infogain=-float('inf')
current_entropy=self._entropy(y)
sample_num=len(y)
```

If the features in X is more than 100, randomly choose 100 features. And we will try to split data by those features later. (If choose all of the features, it may cause memory overflow)

If features in X is less than 100, try all the features to split data

```
if X.shape[1]>100:
    used_feature=np.random.choice(X.shape[1],100,replace=False)
else:
    used_feature = range(X.shape[1])
```

Iterate each feature, sorted the feature values of all sample. If the number of values is more than 10, we only try [10, 20, 30, 40, 50, 60, 70, 80, 90] to be threshold. (prevent memory overflow)

```
for feature_idx in used_feature:
    #try each feature, find the best split
    feature_values = sorted(X.iloc[:, feature_idx].unique())
    #取出 feature_idx 那一列並排序去重
    if len(feature_values)<=1:
        continue
    if len(feature_values)>10:
        thresholds = np.percentile(feature_values, [10, 20, 30, 40, 50, 60,70,80,90])
```

If the number of feature values is not greater than 10, try all middle value between two consecutive feature values as threshold.

```
else:
    thresholds = [(feature_values[i]+feature_values[i+1])/2 for i in
range(len(feature_values)-1)]
```

For all possible threshold, split the data by the threshold. (use \_split\_data func.).

Compute the weighted new entropy of the data being split.

(left\_data\_weight\* left\_data\_entropy + right\_data\_weight\* right\_data\_entropy)

```
for thres in thresholds:
    xl, yl, xr, yr = self._split_data(X,y,feature_idx, thres)
    if len(yl)==0 or len(yr)==0:#doesn't split the data
        continue
```

```

lweight = len(yl)/sample_num
rweight = len(yr)/sample_num
lentropy=self._entropy(yl)
rentropy = self._entropy(yr)
new_entropy = lweight*lentropy+rweight*rentropy

```

Compute information gain for each possible split. (information gain = current \_entropy - new\_entropy). If the information gain is greater the best information gain, update the information gain and update best feature index and best threshold to the current feature index and threshold.

```

if infogain>best_infogain:
    best_infogain=infogain
    best_feature_index=feature_idx
    best_threshold=thres

```

#### 4. **def \_entropy(self, y) → compute the entropy of y**

If y is empty, the entropy is zero.

```

if len(y)==0:
    return 0

```

Counts the occurrence of each class in y. And divide them by total class number to get the probability of each class.

```

_, counts=np.unique(y, return_counts=True)
p=counts/len(y)#probability of each class

```

Compute entropy of y by the following formula:

$$Entropy(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

```

entropy=-np.sum(p*np.log2(p+1e-10))

```

#### 5. **\_predict\_tree(...) → traverse the decision tree by recursively call \_predict\_tree function, and return the predicted value**

If current node is the leaf node, return the value of the node.

```

if tree_node['type']=='leaf':
    return tree_node['value']

```

Else, get the feature index and threshold of the current node. And get the feature value of x(the data to be evaluated)

```

feature_idx=tree_node['feature_idx']
threshold=tree_node['threshold']

```

```
feature_val=x[feature_idx]
```

If the feature value is less than threshold, go to left node by calling `_predict_tree(x, left_subtree)`

```
if feature_val<=threshold:  
    return self._predict_tree(x,tree_node['left'])
```

Else, go to the right node by calling `_predict_tree(x, right_subtree)`

```
else:  
    return self._predict_tree(x, tree_node['right'])
```

6. **def predict(self, X) → call `_predict_tree` function and get the predicted result**  
**For each sample in X, load it into `_predict_tree` function. Store the predicted result in prediction list, and return the results.**

```
predictions=[]  
for i in range(len(X)):  
    x=X.iloc[i]#for each sample in X  
    predict=self._predict_tree(x, self.tree)  
    predictions.append(predict)  
return torch.tensor(predictions)
```

### Experiment:

Validation accuracy of decision tree under different max depth:

1. max depth = 5  
**Validation Accuracy: 0.6979**
2. max depth = 7  
**Validation Accuracy: 0.7776**
3. max depth = 9  
**Validation Accuracy: 0.7754**

When max depth increases from 5 to 7, validation accuracy improves (0.6979 to 0.7776) because the model gains needed complexity to capture important patterns in the data.

When max depth increases from 7 to 9, validation accuracy slightly decreases (0.7776 to 0.7754) because the model begins overfitting, learning noise rather than signal. The result shows that 7 may be the most suitable max depth for decision tree implementation.

3. Kaggle Score:

1. CNN

|    |           |                                                                                   |       |   |    |
|----|-----------|-----------------------------------------------------------------------------------|-------|---|----|
| 26 | 112550184 |  | 0.887 | 3 | 6h |
|----|-----------|-----------------------------------------------------------------------------------|-------|---|----|

2. Decision Tree

|     |           |                                                                                   |       |   |    |
|-----|-----------|-----------------------------------------------------------------------------------|-------|---|----|
| 111 | 112550184 |  | 0.766 | 3 | 6h |
|-----|-----------|-----------------------------------------------------------------------------------|-------|---|----|